

Reinforcement Learning VU (708.006), Summer Term 2026, Assignment 2 Report

Submission Details

Item	Value
Student Name	Joachim Rath
Student ID	51811071
Course	Reinforcement Learning VU (708.006)
Due Date	19.06.2026 23:59

1 Reinforcement Learning in a Grid World

In this part the agent has to control the 4×4 slippery FrozenLake without knowing the transition model. It keeps a tabular $Q(s,a)$ and learns by temporal-difference (TD) control while acting ϵ -greedily. I train each agent for 10 000 episodes with ϵ decaying from 1, and then test the greedy policy ($\epsilon = 0$) for 5 000 episodes. The only reward is 1 for reaching the goal.

1.1 Algorithms

ϵ -greedy means a random action with probability ϵ and $\arg \max$ over a of $Q(s,a)$ otherwise; for testing I set $\epsilon = 0$. The three methods only differ in the TD target used in $Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [\text{target} - Q(s,a)]$. When the next state is terminal the bootstrap term is dropped:

SARSA: $\text{target} = r + \gamma \cdot Q(s', a'), \quad a' \text{ from the policy}$

Q-Learning: $\text{target} = r + \gamma \cdot \max_{a'} Q(s', a')$

Expected SARSA: $\text{target} = r + \gamma \cdot \sum_{a'} \pi(a'|s') \cdot Q(s', a')$

For an ϵ -greedy policy the Expected-SARSA expectation is just $\epsilon \cdot \text{mean}_a Q(s',a) + (1-\epsilon) \cdot \max_a Q(s',a)$.

1.2 Hyperparameters and results

I tuned α and eps_decay separately for each algorithm and γ with a grid search over $\alpha \in \{0.01, 0.05, 0.1, 0.2, 0.5\}$ and $\text{eps_decay} \in \{0.998, 0.999, 0.9993, 0.9995, 0.9997\}$, scoring each setting by the mean test return over 3 seeds. $\alpha = 0.01$ was always too slow. For $\gamma = 0.95$ all three methods reached about 0.78 over a wide range of α . For $\gamma = 1.0$ Q-Learning was unstable: with $\alpha \geq 0.05$ and slow ϵ -decay it often diverged to a useless policy (return 0), and only $\alpha = 0.05$ worked, while SARSA and Expected SARSA

stayed stable. Table 1 shows the values I used and the returns I got; all six are close to Policy Iteration (0.786 for $\gamma = 0.95$, 0.826 for $\gamma = 1.0$).

Algorithm	γ	α	eps_decay	train R	test R	PI test
SARSA	0.95	0.05	0.9993	0.49	0.77	0.786
Q-Learning	0.95	0.10	0.9990	0.55	0.78	0.786
Expected SARSA	0.95	0.10	0.9993	0.48	0.78	0.786
SARSA	1.0	0.10	0.9997	0.23	0.82	0.826
Q-Learning	1.0	0.05	0.9993	0.51	0.77	0.826
Expected SARSA	1.0	0.10	0.9997	0.24	0.82	0.826

Table 1: Chosen hyperparameters (seed 0) and resulting mean training / test return, with the Policy-Iteration reference.

Figures 1–6 show the training and test reward, the learned policy (white arrows) and the value function for each case.

FrozenLake | SARSA with $\gamma = 0.95$



Figure 1: SARSA, $\gamma = 0.95$.

FrozenLake | Q-Learning with $\gamma = 0.95$

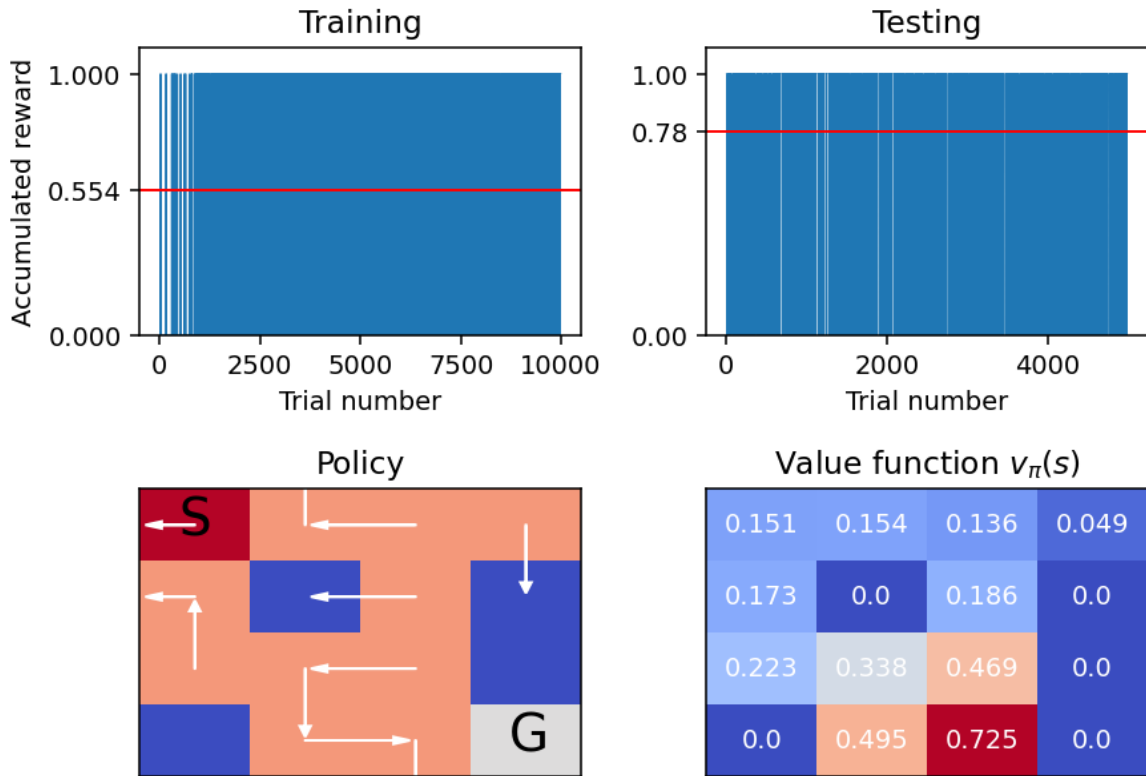


Figure 2: Q-Learning, $\gamma = 0.95$.

FrozenLake | Expected SARSA with $\gamma = 0.95$

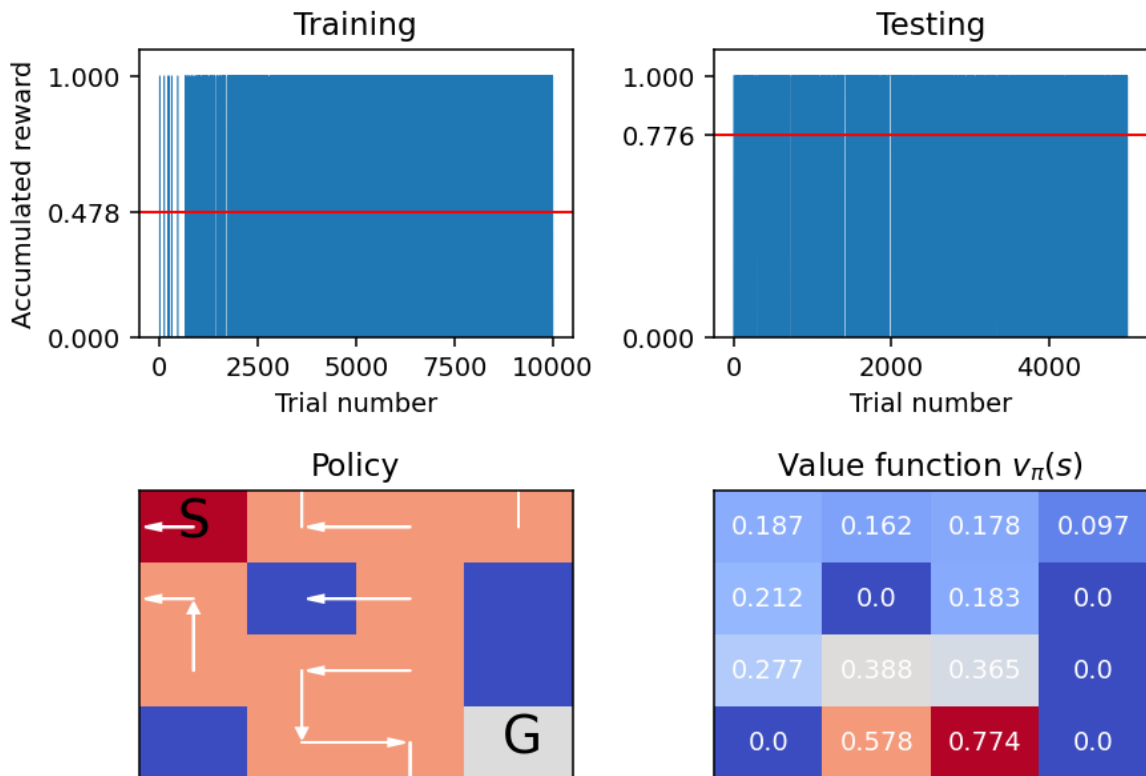


Figure 3: Expected SARSA, $\gamma = 0.95$.

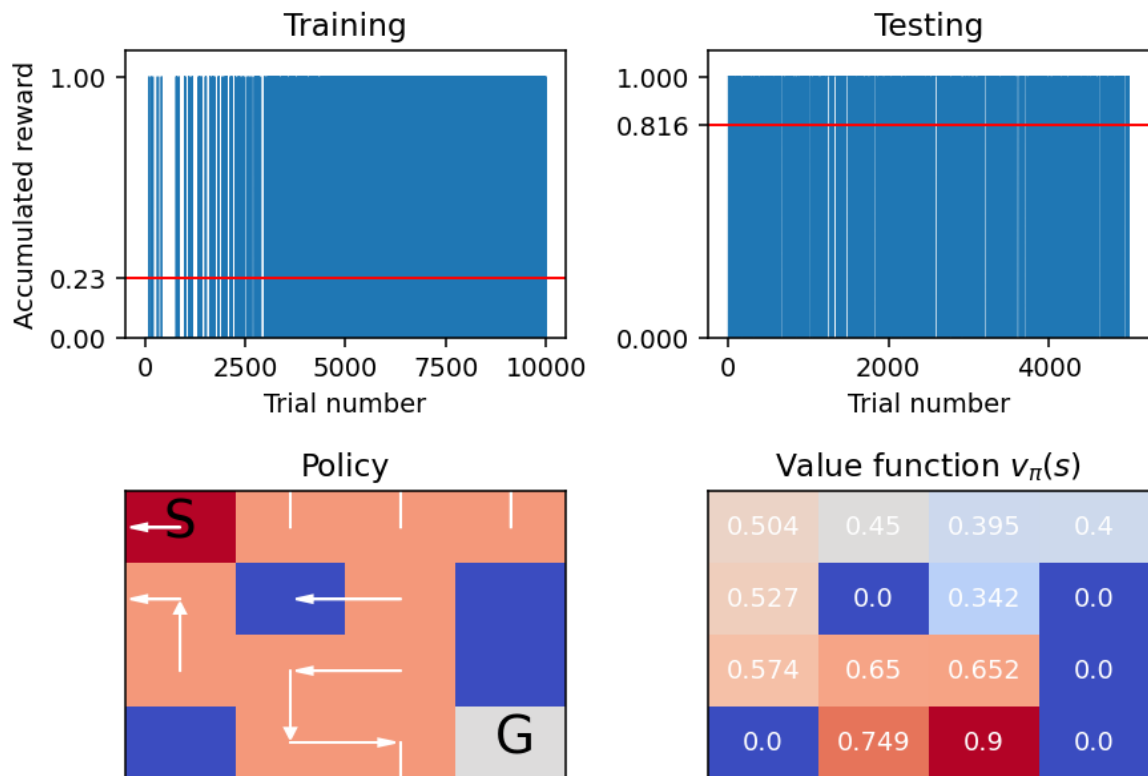
FrozenLake | SARSA with $\gamma = 1.0$ 

Figure 4: SARSA, $\gamma = 1.0$.

FrozenLake | Q-Learning with $\gamma = 1.0$

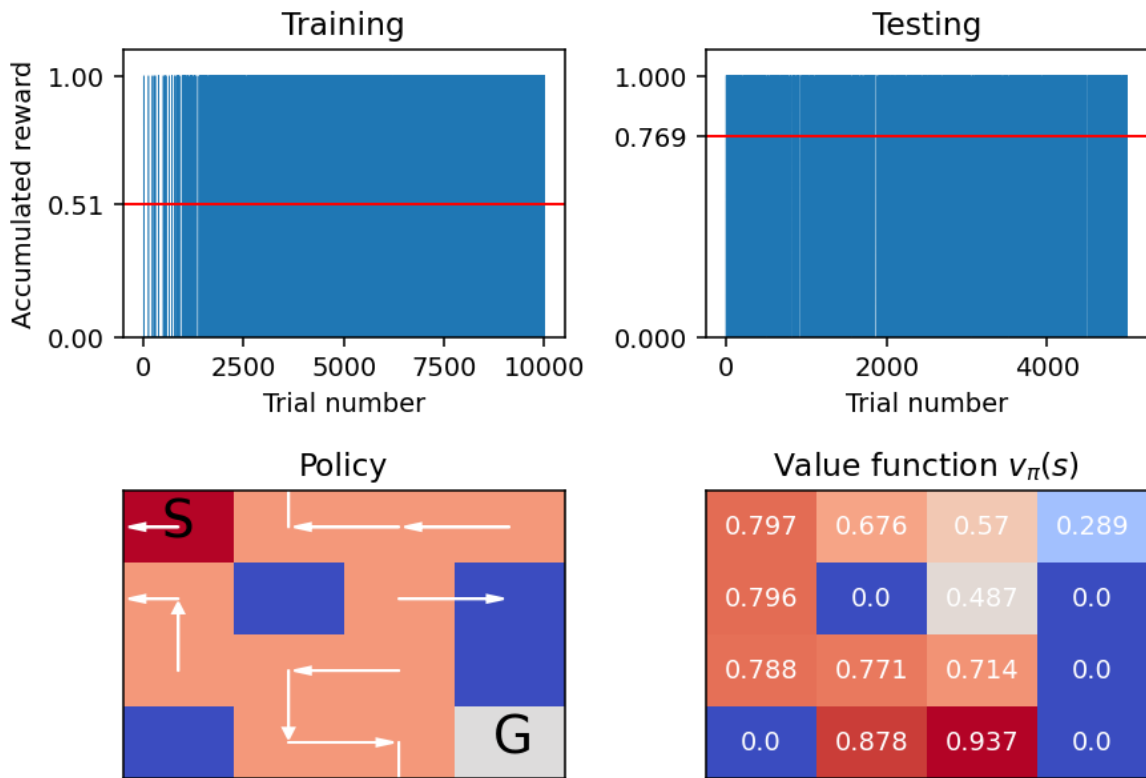


Figure 5: Q-Learning, $\gamma = 1.0$.

FrozenLake | Expected SARSA with $\gamma = 1.0$

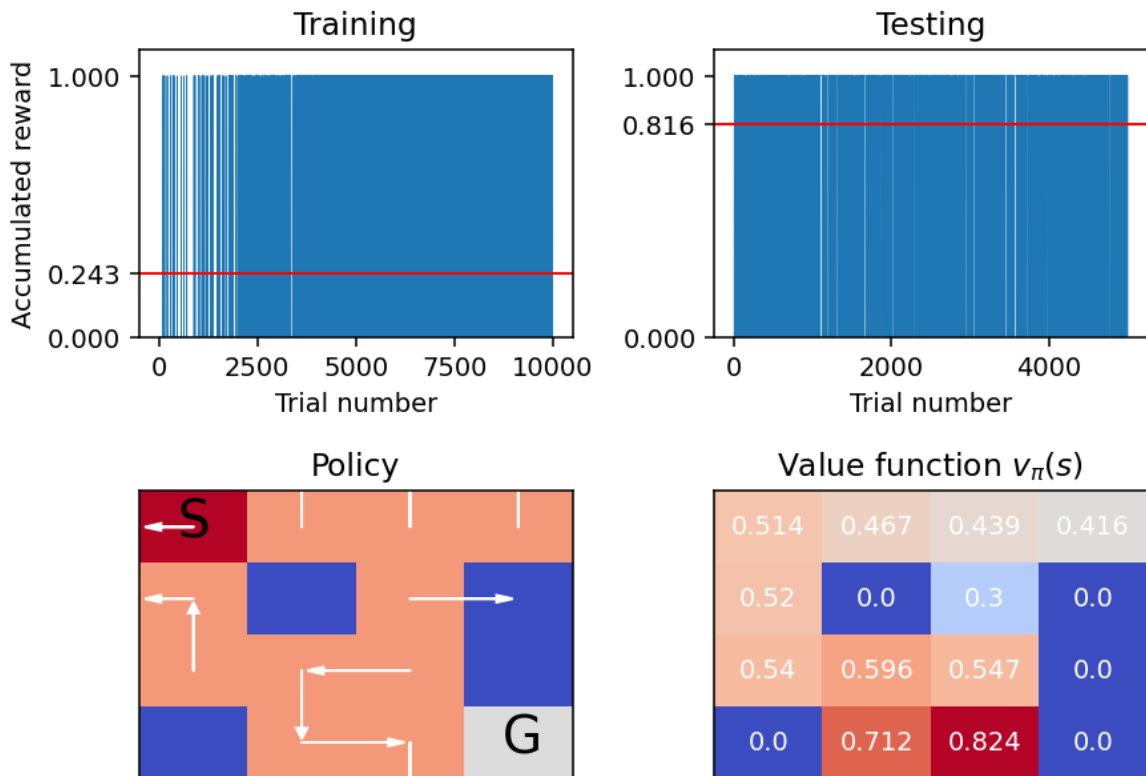


Figure 6: Expected SARSA, $\gamma = 1.0$.

1.3 Comparison with Policy Iteration

All three methods find essentially the same greedy policy as Policy Iteration and reach a similar success rate. The typical slippery trick is visible: next to a hole the policy points away from it, often into a wall, so a random sideways slip cannot fall in. The value functions agree well at $\gamma = 0.95$. At $\gamma = 1.0$ they differ (Table 2): Q-Learning still matches v^* because it learns greedy values off-policy, while SARSA and Expected SARSA underestimate the states far from the goal. They are on-policy and evaluate the exploring ϵ -greedy policy, and without discounting the value spreads only slowly from the single goal reward, so far-away cells stay too low after 10 000 episodes.

$v(s)$ at $\gamma = 1.0$	start (0,0)	left-col (2,0)	left-of-goal (3,2)
Policy Iteration (v^*)	0.824	0.824	0.941
Q-Learning	0.797	0.788	0.937
SARSA	0.504	0.574	0.900
Expected SARSA	0.514	0.540	0.824

Table 2: Value of three cells at $\gamma = 1.0$. Near the goal all agree; far from it only Q-Learning stays close to v^* .

2 Deep Q-Learning for Atari Breakout

Here I train a DQN (Mnih et al., 2015) on Atari Breakout with a convolutional network, a fixed target network and an experience-replay buffer. Frames are turned to grayscale, resized to 84×84 and the last 4 are stacked, so a state is a $4 \times 84 \times 84$ tensor; there are 4 actions.

2.1 Network and fixed target network

The online network has three convolutional layers (32 filters 8×8 stride 4, 64 filters 4×4 stride 2, 64 filters 3×3 stride 1, each with ReLU), then a 512-unit dense layer and a final layer with one output per action (about 1.7M parameters). A second network with the same shape is the target network. Its weights are frozen and only copied from the online network every `sync_target` frames. The target value is $y = r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a')$, and I minimise the MSE to $Q_{\text{online}}(s, a)$ with Adam. The target network keeps this regression target fixed for a while instead of letting it move on every update, which stops the training from oscillating. The `burn_in_phase` is the number of frames at the start where the agent only fills the buffer ($\epsilon = 1$, no updates, no ϵ -decay), so the first minibatches are not drawn from an almost empty, highly correlated buffer.

2.2 Experience replay memory

The buffer is a deque of past transitions and `sample()` takes a uniform random minibatch. The reason for it is that SGD assumes the samples are i.i.d. (independent and identically distributed), which is violated if you learn from transitions in the order they happen: consecutive transitions are highly correlated, and the data distribution keeps changing as the policy changes (non-stationary). Sampling randomly from a large buffer

mixes many episodes and policies, so the minibatches are roughly independent again, and every transition can be reused several times.

2.3 Results

I trained on an Apple-M2 laptop (8 GB, MPS backend), which is far too little for Breakout (the original DQN uses about 50M frames), so this is only a short demonstration. In about 40 minutes the agent ran 1576 episodes (144,217 steps) and ϵ reached 0.1. The mean episode reward went up from 2 to 10 (best episode 21), so it clearly learns to play better than random, which scores about 2, and the loss stays stable after the burn-in (Figure 7). With a GPU and a larger buffer the same code would reach higher scores.

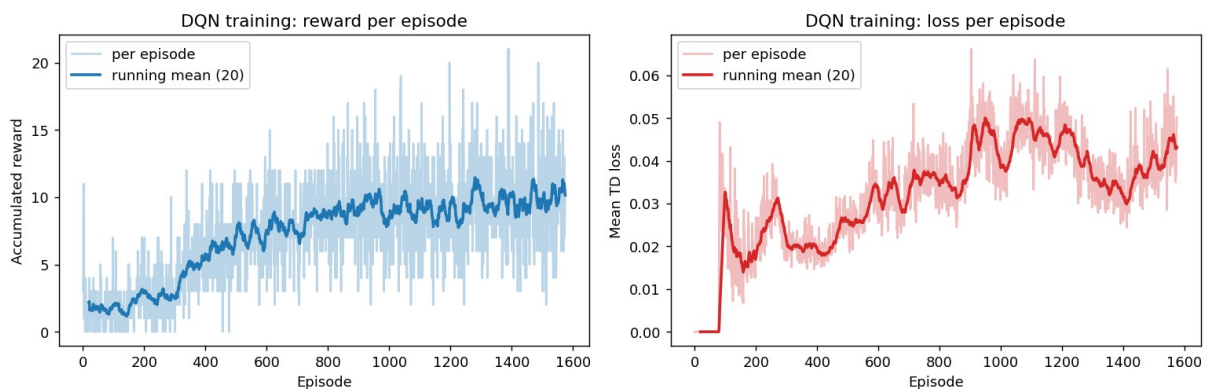


Figure 7: DQN on Breakout — reward per episode (left) and TD loss (right), each with a running mean.

3 Use of LLMs

I used Claude (Anthropic) through the Claude Code assistant. It helped me write the TD updates and the ϵ -greedy policy, run and interpret the hyperparameter search (including why Q-Learning is unstable at $\gamma = 1.0$), fill in the DQN skeleton (network, target sync, replay sampling, optimiser) and adapt the code to my 8 GB laptop. It also noticed that the agent was being created without the `run_as_ddqn` argument it expects, and that the per-episode loss divides by the step counter. The main limitations were that the given skeleton needed those fixes to run, and that I could not train Breakout to a high score on my hardware, so the DQN part stays a short demonstration. I went through the code and the reasoning myself and understand how it works.